Responsive design

Responsive design means a page can adapt when the available space changes. It is about making the same content work in a narrow browser, a wide monitor, and the sizes in between.

A responsive page usually needs a few habits:

- Let the main container shrink instead of forcing a fixed width.
- Let images and media fit their parent.
- Start with a simple layout that works on small screens.
- Add wider layouts only when there is enough room.
- Test by resizing the browser, not by guessing.

Start with the narrow screen first

This lesson uses a small workshop page. The page has a headline, an image, a pair of action links, and three feature cards.

The important move is the order: first make the page comfortable in one column. Then add a wider layout with a media query.

Use this as `index.html`:

html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Responsive practice</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="page">
    <section class="hero-card">
      <div class="hero-copy">
        <p class="eyebrow">CSS practice</p>
        <h1>Design a page that adapts</h1>
        <p class="summary">A responsive page should stay readable and usable as the br

        <div class="actions">
          <a class="primary-action" href="/start">Start practice</a>
          <a class="secondary-action" href="/notes">Read notes</a>
        </div>
      </div>

      <img class="hero-image" src="https://dummyimage.com/960x640/dbeafe/1e3a8a.png?te
    </section>

    <section class="feature-list" aria-label="Responsive design habits">
      <article class="feature-card">
```

```
          <h2>Fluid containers</h2>
          <p>Let the page use available space without forcing one fixed width.</p>
        </article>

        <article class="feature-card">
          <h2>Flexible media</h2>
          <p>Let images shrink with their parent so they do not overflow.</p>
        </article>

        <article class="feature-card">
          <h2>Useful breakpoints</h2>
          <p>Change the layout when the content needs more room, not because of a device
        </article>
      </section>
    </main>
  </body>
</html>
```

The `meta viewport` line in the `head` matters:

html

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

It tells mobile browsers to use the real device width as the layout width. Without it, a phone browser may pretend the page is much wider than the screen, then shrink the whole page down. That makes responsive CSS harder to reason about.

Then add the base CSS:

Previous Lesson Next Lesson

```
* {
  box-sizing: border-box;
}

body {
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f9fafb;
}

.page {
  max-width: 960px;
  margin: 32px auto;
  padding: 0 16px;
}

.hero-card,
.feature-card {
  background-color: white;
  border: 1px solid #e5e7eb;
  border-radius: 8px;
  padding: 24px;
}

.hero-card {
  margin-bottom: 20px;
}

.eyebrow {
  color: #2563eb;
  font-size: 14px;
  font-weight: 700;
}

h1,
```

```
h2,
p {
  margin-top: 0;
}

h1 {
  font-size: 36px;
  line-height: 1.1;
}

.summary,
.feature-card p {
  color: #4b5563;
  line-height: 1.6;
}

.actions {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
  margin: 20px 0;
}

.primary-action,
.secondary-action {
  border-radius: 6px;
  font-weight: 700;
  padding: 10px 14px;
  text-decoration: none;
}

.primary-action {
  color: white;
  background-color: #2563eb;
}

.secondary-action {
  color: #2563eb;
  border: 1px solid #bfdbfe;
}

.hero-image {
  display: block;
  width: 100%;
  height: auto;
  border-radius: 8px;
}

.feature-card {
  margin-bottom: 16px;
}
```

Open the page and make the browser narrow. The page should still be readable because the base layout is a single column.

Use a fluid container

The following `.page` rule is the first responsive habit:

css

```
.page {
  max-width: 960px;
  margin: 32px auto;
  padding: 0 16px;
}
```

`max-width: 960px` says the page should not grow wider than `960px`. That keeps long lines from becoming hard to read on wide screens.

The page does not set a fixed `width`. That means it can shrink naturally when the viewport is narrower than `960px`.

`margin: 32px auto` adds space above and below the page, and centers it horizontally when there is extra room.

`padding: 0 16px` gives the content breathing room on small screens so text and cards do not touch the edge of the viewport.

> ⚠ **Avoid fixed page widths** - A rule like `width: 960px` can force horizontal scrolling on small screens. Prefer a flexible container with `max-width` and side padding.

Make images flexible

Images have a natural size from the file. If that natural size is wider than the screen, the image can overflow.

This rule keeps the image inside its parent:

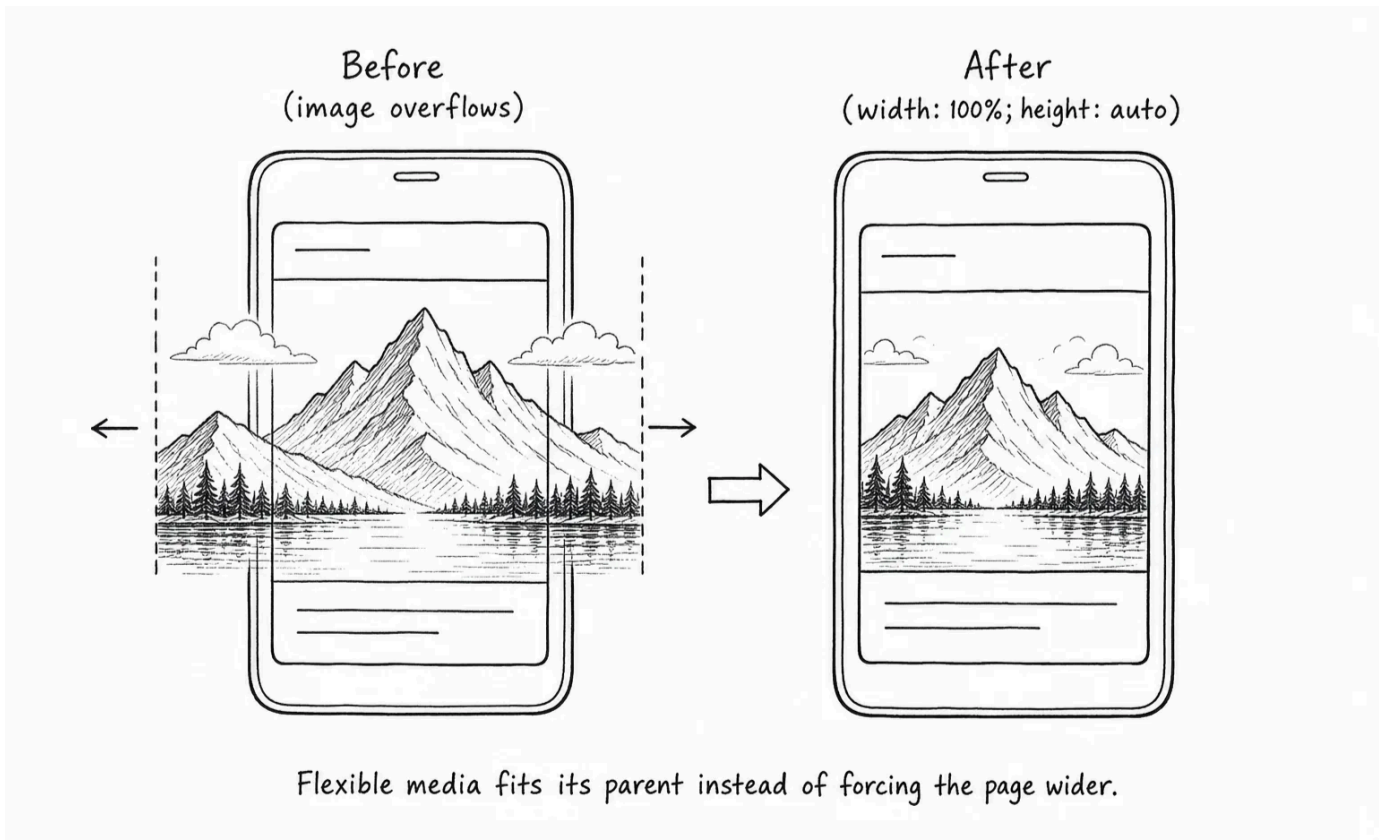
```css
.hero-image {
  display: block;
  width: 100%;
  height: auto;
  border-radius: 8px;
}
```

`width: 100%` makes the image fill the available width of its parent.

`height: auto` keeps the image's natural proportions, so the browser does not stretch it.

`display: block` removes inline image spacing behavior, which you saw in the images lesson.

The `border-radius` is visual polish that rounds the image corners so the image matches the card shape.



Add a breakpoint when the layout needs one

The single-column layout works well on narrow screens. On wider screens, the hero image has enough room to sit beside the text.

That is what a media query is for. A media query applies CSS only when a condition is true.

Add this after the base CSS:

```css
@media (min-width: 720px) {
  .hero-card {
    display: flex;
    align-items: center;
    gap: 24px;
    justify-content: space-between;
  }

  .hero-copy,
  .hero-image {
    flex: 1;
    max-width: 300px;
  }
}
```

`@media (min-width: 720px)` means "use these rules when the viewport is at least `720px` wide."

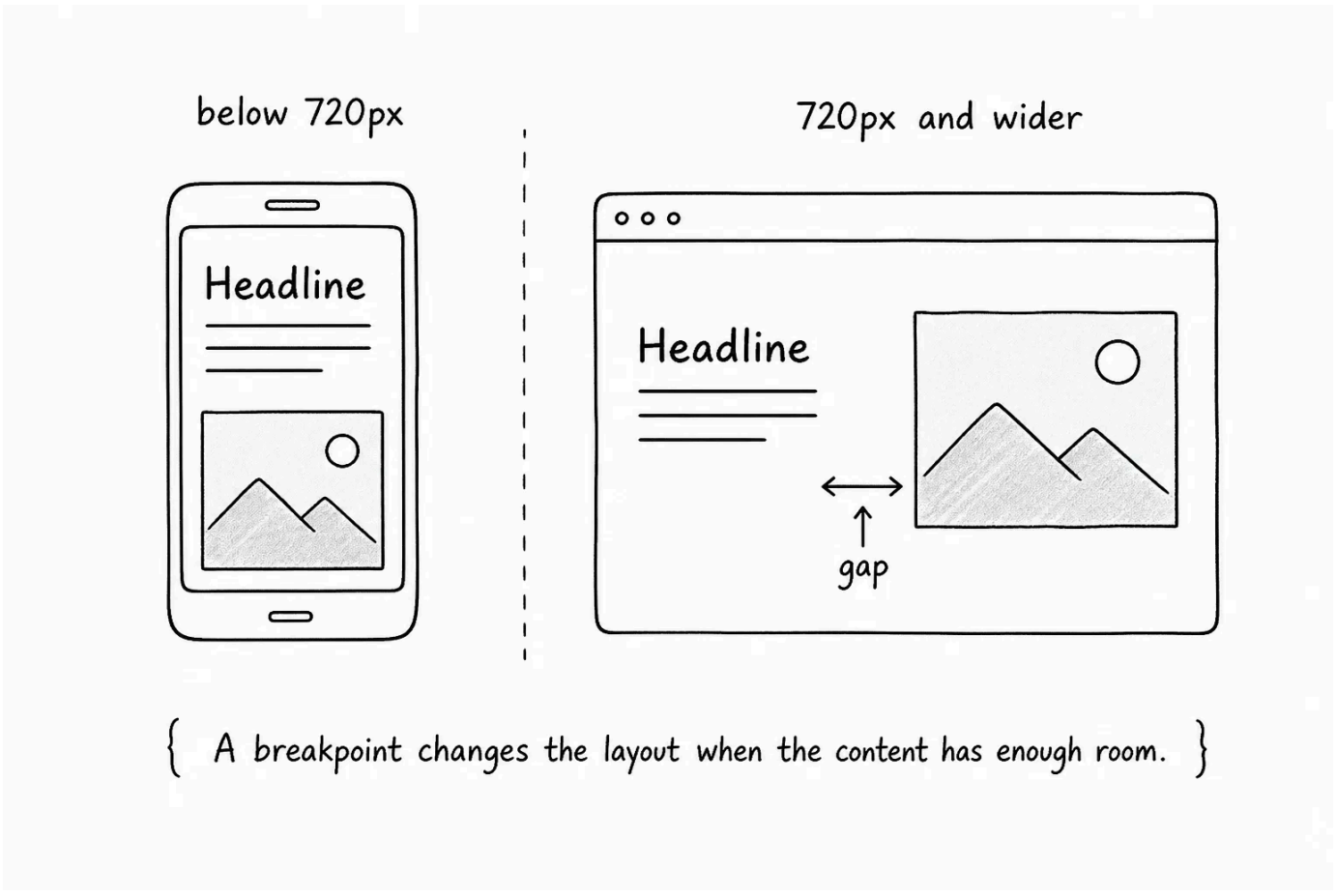
This is a mobile-first breakpoint: the normal CSS handles small screens first, and the media query adds a wider layout only when there is enough space.

Inside the media query, `.hero-card` becomes a flex container. Its direct children, `.hero-copy` and `.hero-image`, sit in a row.

`align-items: center` lines the text and image up vertically, and `gap: 24px` creates space between them.

`flex: 1` lets the text area and image share the available row space.

Resize the browser. Below `720px`, the hero should stack; at `720px` and above, it should become a two-column hero.



Use breakpoints for content, not devices

A breakpoint is the point where your CSS changes because the layout needs a different shape.

The breakpoint should come from the content. If the hero feels cramped at `680px`, do not force it into two columns. If it has enough room at `720px`, that may be a good breakpoint.

Avoid thinking only in device names like "phone", "tablet", and "desktop". Device sizes change, and content needs are easier to reason about.

Resize until the layout complains - To choose a breakpoint, resize the browser slowly. Add a media query where the content starts to feel cramped, stretched, or awkward.

Make the cards responsive too

The feature cards stack on small screens, which is a good default. On wider screens, they can sit in a row.

Add this to the same media query:

```css
@media (min-width: 720px) {
  .hero-card {
    display: flex;
    align-items: center;
```

```
    gap: 24px;
    justify-content: space-between;
  }

  .hero-copy,
  .hero-image {
    flex: 1;
    max-width: 300px;
  }

  .feature-list {
    display: flex;
    gap: 16px;
  }

  .feature-card {
    flex: 1;
    margin-bottom: 0;
  }
}
```

The `.feature-list` becomes a flex container only on wider screens. Its direct children, the `.feature-card` elements, become flex items.

`gap: 16px` owns the space between the cards.

`flex: 1` lets each card share the row evenly.

`margin-bottom: 0` removes the stacked-card spacing from the small-screen version. On wide screens, `gap` handles the spacing between cards.

This is a common responsive pattern: stack by default, then switch to a row when there is enough room.

Check text size and spacing

Responsive design is not only layout. Text and spacing can also feel different at different viewport sizes.

The base `h1` is large enough for the small layout:

css

```
h1 {
  font-size: 36px;
  line-height: 1.1;
}
```

On wider screens, the heading can grow a little because it has more room:

css

```
@media (min-width: 720px) {
  h1 {
    font-size: 48px;
  }
}
```

Do this carefully. Bigger text can make a wide layout feel stronger, but it can also create awkward wrapping. Resize the browser after changing heading size and check where the lines break.

You do not need a media query for every property. Add one only when the current value stops working at a different size.

Test with DevTools

DevTools can simulate different viewport widths.

Open DevTools and look for the device toolbar or responsive mode. Drag the viewport wider and narrower, and watch for these problems:

- Horizontal scrolling.
- Text touching the edge of the screen.
- Images overflowing their cards.
- Buttons becoming too cramped to read or tap.
- Lines of text becoming too long on wide screens.
- Layout changes that happen too early or too late.

Responsive design is a testing habit as much as a CSS technique.

Put the responsive rules together

Here are the responsive pieces you added:

```css
.page {
  max-width: 960px;
  margin: 32px auto;
  padding: 0 16px;
}

.hero-image {
  display: block;
  width: 100%;
  height: auto;
  border-radius: 8px;
}

@media (min-width: 720px) {
  .hero-card {
    display: flex;
    align-items: center;
    gap: 24px;
    justify-content: space-between;
  }

  .hero-copy,
  .hero-image {
    flex: 1;
    max-width: 300px;
  }

  .feature-list {
```

```
    display: flex;
    gap: 16px;
  }

  .feature-card {
    flex: 1;
    margin-bottom: 0;
  }

  h1 {
    font-size: 48px;
  }
}
```

The pattern is simple: write the small-screen layout first, then add media queries when wider screens have enough room for a different layout.

Try it

Test the page by resizing it on purpose:

- Change `.page` from `max-width: 960px;` to `width: 960px;`. Make the browser narrow and notice the horizontal scrolling. Then change it back.
- Remove `width: 100%;` from `.hero-image` and check whether the image can overflow.
- Change the media query from `min-width: 720px` to `min-width: 900px`. Notice when the hero switches to two columns.
- Change the media query to `min-width: 500px`. Decide whether the two-column layout happens too early.
- Remove `.feature-card { margin-bottom: 0; }` inside the media query. Notice whether old stacked spacing still affects the row.
- Add a fourth feature card and check whether the row still feels comfortable.
- Use DevTools responsive mode and test at several widths, not just one.

The goal is not to memorize a perfect breakpoint. The goal is to make the page respond when the content needs a different amount of space.

Click to Reveal the Answer

Already Know that | Didn't Know that | Skip Question

LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →